MBEYA UNIVERSITY OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF INFORMATION COMMUNICATION AND TECHNOLOGY

MODULE NAME: MOBILE APPLICATION DEVELOPMENT

TASK: PROJECT REPORT

LECTURER'S NAME: MR RUKOIJO A

ACADEMIC YEAR: 2025/2026.

MODULE CODE IT 8203

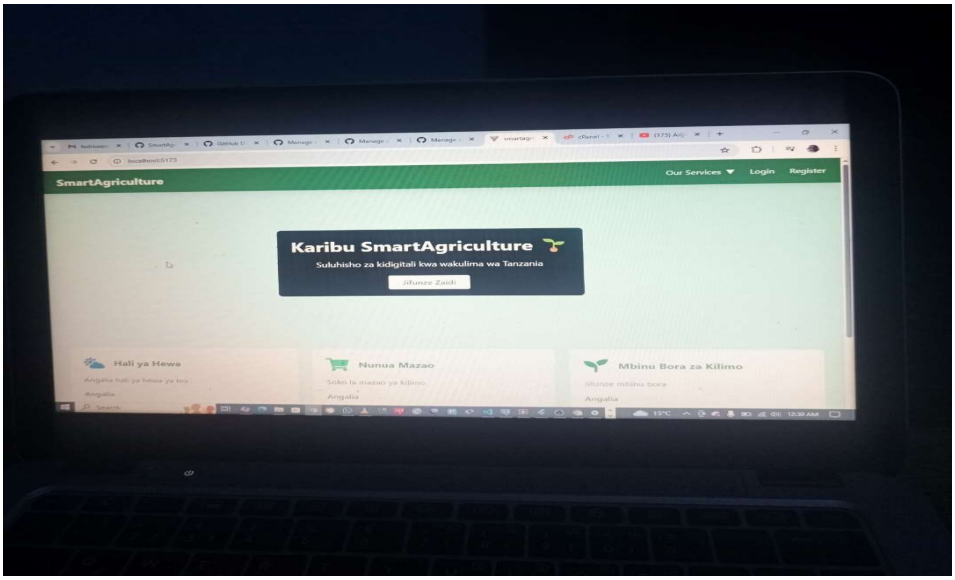
COURSE NAME: BACHELOR OF SCIENCE IN INFORMATION AND COMMUNICATION TECHNOLOGY(BScICT)

ACADEMIC LEVEL: UQF8 SECOND YEAR

GROUP MEMBERS

NO	NAME	REG NUMBER	CA NUMBER
1	FEDRICK EMANUELY PETRO	24101133370032	CA/BScICT/25/13469
2	EMANUEL FRANK MOSHA	24101133370020	CA/BScICT/25/14594
3	KASUKA SAYI KUSHAHA	24101133370097	CA/BScICT/25/11246
4	PAUL SAFARI YANGOLE	24101133370098	CA/BScICT/25/11254

SMART AGRICULTURE



Introduction & Problem Definition

Introduction

Agriculture plays a vital role in the economy, especially in developing countries where a large percentage of the population depends on farming for income and food supply. However, many farmers still rely on traditional methods to manage farming records and report agricultural challenges

Smart agriculture is a web-based application designed to digitize the process of reporting and managing farming problems. The system provides a centralized platform where farmers can securely register, log in, and manage their agricultural issues online.

The system improves record keeping, enhances communication efficiency, and ensures that data is securely stored in a structured database.

Problem Definition

Farmers face multiple operational and technological challenges:

1.Poor record keeping; Many farmers use notebooks, which can be lost or damaged.
2.No centralized reporting system;There is no unified digital platform for problem tracking.
3.Delayed expert support;Without digital communication, assistance is slow.
4.Data insecurity;Paper records are not secure.
5.Limited digital integration; Most small-scale farmers lack access to management systems.

Objectives of the System

General Objective

To develop a secure and user-friendly web application that enables farmers to report and manage agricultural problems digitally.

Specific Objectives

a.To implement a secure authentication system
b.To create a database for storing farming problems
c.To implement CRUD functionality

d.To design a responsive user interface
e.To integrate frontend, backend, and database systems

1.2 Requirements (Functional & Non-Functional)

Functional Requirements

-User Registration
- User Authentication
- Session Management
- Problem Creation
- Problem Viewing
- Problem Updating
- Problem Deletion
- Logout

Non-Functional Requirements

a.Security:
- Password encryption
- Session protection
- Secure database credentials

b.Performance:
- Fast response time
- Efficient database queries

c.Usability:
- Simple and intuitive interface

d.Scalability:

- The system should allow future upgrades

e.Reliability:
- System must operate without frequent crashes

f.Responsiveness:
- Compatible with mobile and desktop devices

System Design & Architecture

The system follows a Client-Server Architecture.
Architecture Flow:
1. The user interacts with the frontend (browser).
2. The frontend sends HTTP requests to the Express server.
3. The backend processes requests.
4. The backend communicates with SQL.
5. The database returns data to the server.
6. The server sends a response to the user interface.

Technology Stack

Frontend Layer:
- HTML (Structure)
- CSS (Styling)
- JavaScript (Interactivity)

Backend Layer:

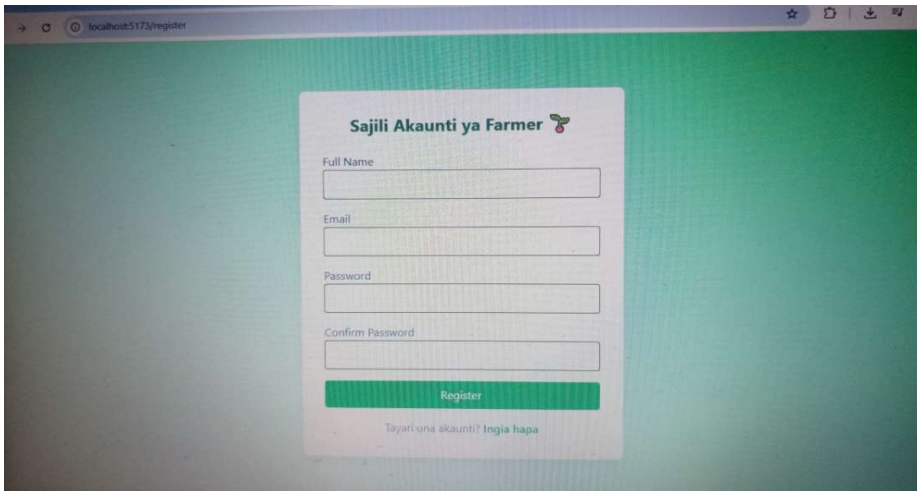
-Django
- Node.js (Runtime environment)
- Express.js (Web framework)

Database Layer:

-postgree SQL
-SQL Database)
- Mongoose (ODM)

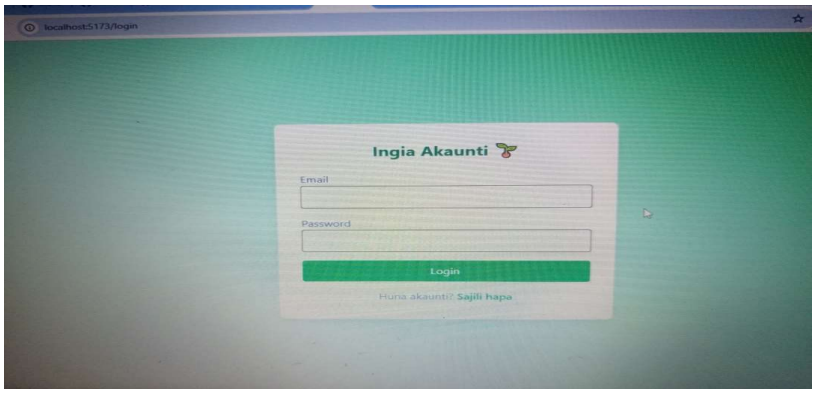
Database Design

Users Collection:
1._id (ObjectId)
2.name (String)
3.email (String)
4.password (String - Hashed)



Problems Collection:
1. _id (ObjectId)
2.title (String)
3.description (String)

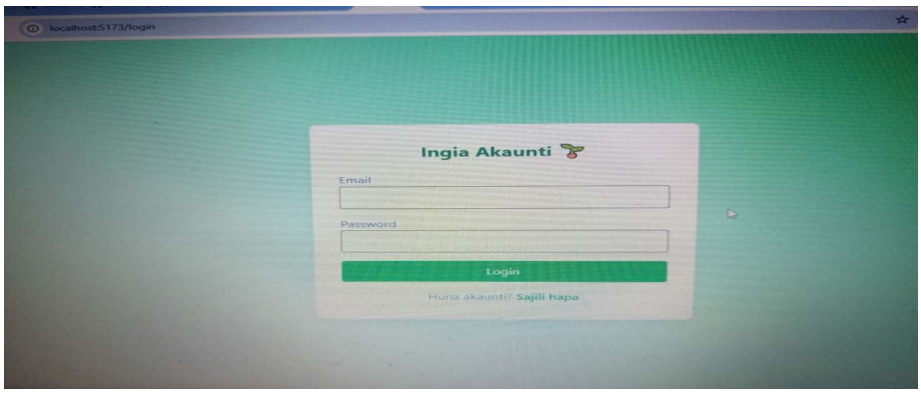
4.userId (ObjectId)
5.createdAt (Date)



1.4 Security & Authentication Explanation

Authentication Process:

1. User registers with email and password.
2. Password is hashed using bcrypt.
3. On login, entered password is compared with stored hashed password.
4. If valid, a session is created.
5. Session is destroyed on logout.

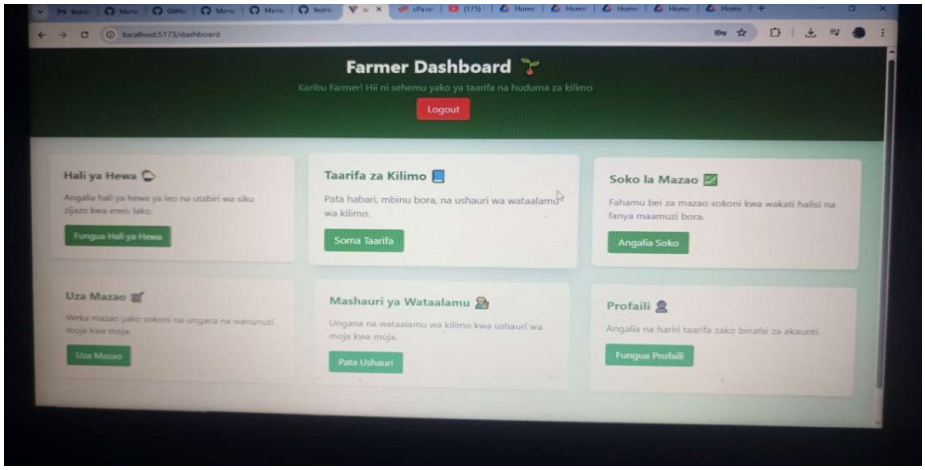


Security Measures:

- Password Hashing
- Express-Session

- Environment Variables (.env)
- Route Protection

Results Achieved:
1.Registers users securely
2.login credentials
3.Connects to MongoDB Atlas
4.Stores and retrieves data
5.Performs full CRUD operations
6.Maintains user sessions



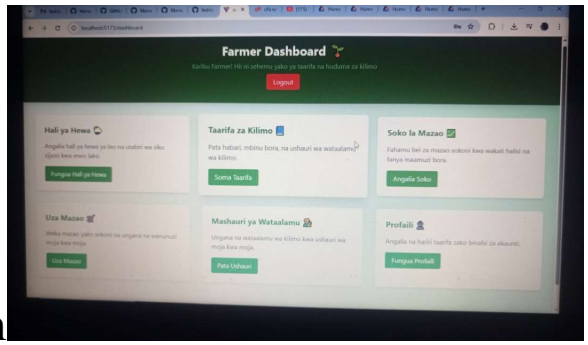
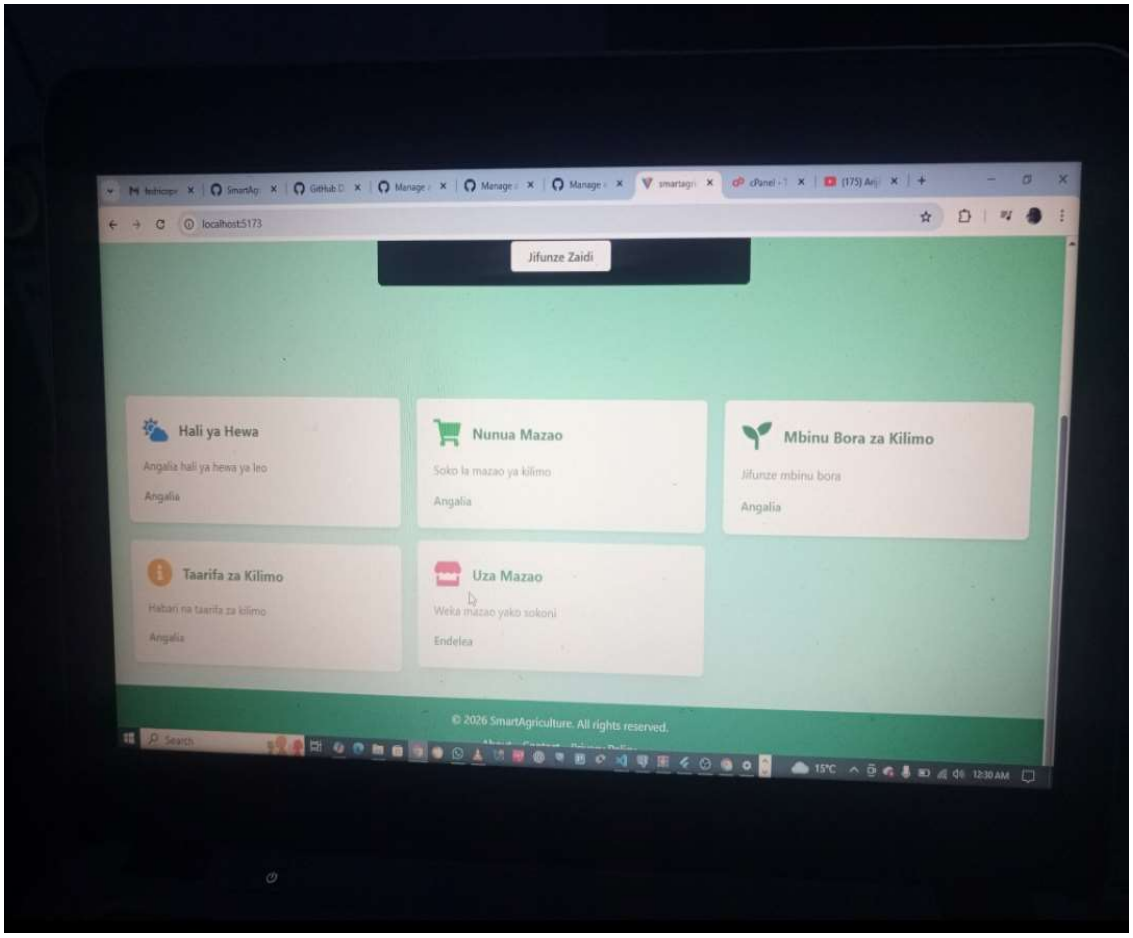
System Testing:
- Manual testing of all forms
- Database verification
- Login validation testing
- Error handling testing

Conclusion:
Smart agriculture System successfully provides a digital platform for managing agricultural problems. It enhances

data security, improves record keeping, and ensures efficient problem management.

Future Improvements:

- Admin dashboard
- Notification system
- AI crop disease detection
- SMS integration



- Mobile application version